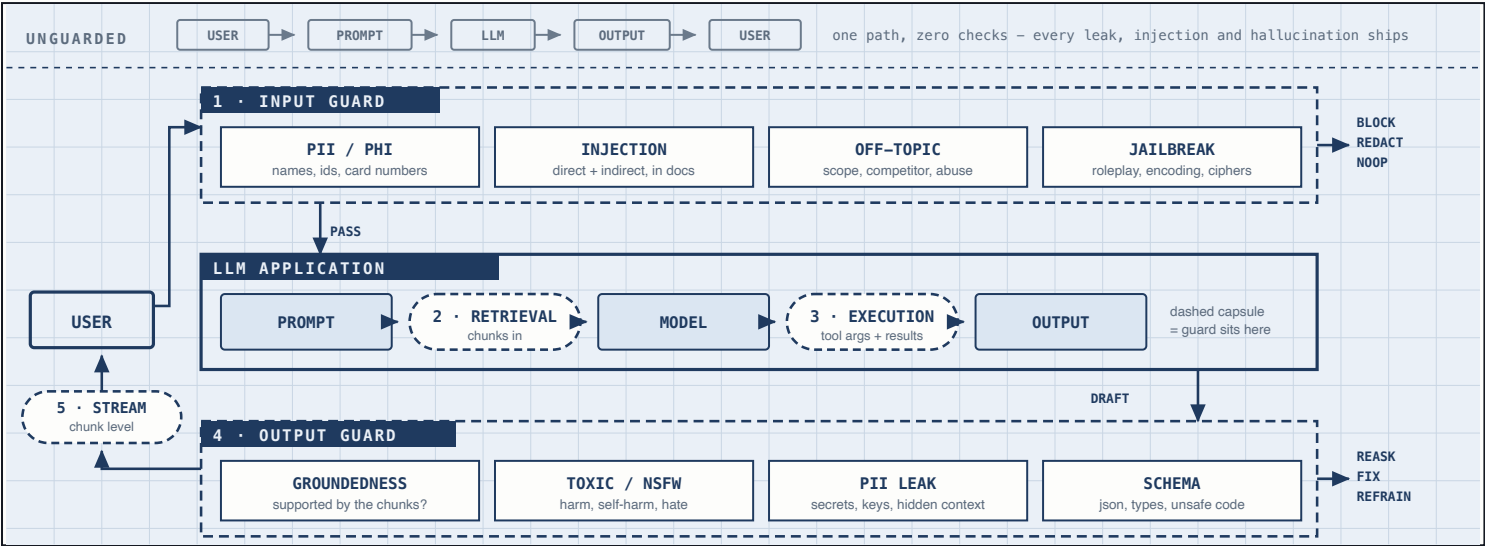


Where Guardrails Go in an AI Application

Five places a guard can sit, what each one costs, and the numbers vendors do not lead with · 2026-08-20



THREE THINGS, OFTEN CONFUSED

- **Alignment** - training time. In the weights. The provider's call, not yours.
- **Guardrail** - request time. In the path. Can *change* or *stop* the response.
- **Eval** - build time. Measures. Stops nothing.

Alignment lowers the base rate, guardrails cap the tail, evals tell you whether either works. A system prompt does none of the three. ◦

FIVE PLACES A GUARD SITS

NeMo's rail taxonomy is the most complete public one:

- **1 Input** - the user turn.
- **2 Retrieval** - chunks before they enter the prompt. Indirect injection lives here.
- **3 Execution** - tool args out, tool results back. The only guard that stops real-world damage.
- **4 Output** - the draft before the user sees it.
- **5 Stream** - chunk level, sliding window (NeMo: 200-token chunks, 50 carried for context).

Dialog rails - should this action run at all - are the sixth, and the layer most stacks lack.

THE TOOL SURFACE IS BLIND

Bedrock guardrails do **not** inspect `toolUse.input`, `toolResult`, or `toolSpec`. PII in a model-generated tool call is neither blocked nor masked. Hence `InvokeGuardrailChecks` (June 2026): a score-only check you call yourself before a tool runs. OpenAI Agents SDK mirrors the trap - input guardrails run only for the **first** agent, output guardrails only for the **last**. Every intermediate handoff is bare.

ORDER GUARDS BY COST

word filter / regex	~0ms	free
PromptGuard 2 22M	19ms	.942 AUC
PromptGuard 2 86M	92ms	.995 AUC
CodeShield tier 1	60ms	96% prec
CodeShield tier 2	300ms	
grounding / LLM judge	full generation	

Anthropic's two-stage cascade - light classifier on all traffic, ensemble only on escalation - cut overhead from **23.7% to about 1%**, a 40x saving, while refusals on harmless queries fell to 0.05%.

Cheap and deterministic first, independent guards in parallel. Never put an LLM judge in front of a regex.

WHAT GUARDS ACTUALLY COST

Bedrock, per 1,000 text units (~1,000 chars):

Content filters	\$0.15
Denied topics	\$0.15
Sensitive info (PII)	\$0.10
Contextual grounding	\$0.10
Automated reasoning	\$0.17
Word filters, regex	free

Stack four and you are at **\$0.50 per 1,000 text units before a single model token** - on short chat turns, more than the model. Spend the free filters first.

ON FAIL: THE REAL ACTION SET

Guardrails AI enumerates it best: `EXCEPTION` (fail closed), `REASK`, `FIX`, `FIX_REASK`, `FILTER` (drop the failing field, structured data only), `REFRAIN` (return nothing), `NOOP` (log only), `CUSTOM`.

Ship every new guard on **NOOP** - Bedrock's `action: NONE` returns score, confidence and threshold; Model Armor calls it `Inspect Only`. Cap `REASK` at 1-2: uncapped retries are OWASP LLM06 Unbounded Consumption, and they loop hardest on adversarial input.

THE TOOLBOX

Tool	Shape
Guardrails AI 0.11	lib + REST server, 65 hub validators
NeMo Guardrails 0.23	Colang; only stack with dialog + retrieval + streaming rails
Llama Guard 4, Prompt Guard 2, CodeShield	open weights, Llama 4 licence
Bedrock Guardrails	managed; formal-verification grounding
Azure Content Safety	Prompt Shields; grounding with correction
Google Model Armor	GA, model-agnostic, project floor settings
Presidio	PII; community-owned now, left Microsoft
LLM Guard	35 scanners; no release since May 2025 - check maintenance

NUMBERS NOBODY LEADS WITH

- **Llama Guard 4 12B**, English: **69% recall at 11% FPR** - it misses ~3 in 10 unsafe items and flags ~1 in 9 safe ones. Multilingual recall drops to **43%**.
- **Prompt Guard 1** scored .987 AUC but only **21.2% TPR at 1% FPR**. v2 86M scores 97.5%. AUC alone is nearly useless - ask for TPR at your operating FPR.
- **LlamaFirewall on AgentDojo**: attack success 17.6% to 1.8%, utility 0.477 to 0.427. Ninety percent of the attack gone for ~5 points of capability. The honest price.

DOCUMENTED TRAPS

- Bedrock logs blocked content as **plain text**. PII masking does not apply to invocation logs, and the trace `match` field holds the original value by *design*.
- Bedrock contextual grounding **does not support conversational use cases**. Teams ship it on RAG chat anyway.
- `ASYNCHRONOUS` streaming returns chunks before scanning, and PII masking is silently unavailable in that mode.
- English-only cliffs: Azure groundedness, protected material, custom categories; Bedrock Classic tier. A suite validated in English is not validated.

STANDARDS TO MAP AGAINST

- **OWASP GenAI LLM Top 10, 2026**: LLM01 injection, LLM02 sensitive info, LLM03 excessive agency, LLM06 unbounded consumption, LLM08 hidden context exposure, LLM10 output handling - six of ten are guardrail territory. ◦
- **NIST AI 600-1** - 12 GenAI risk categories, 200+ actions.
- **ISO/IEC 42001** - 38 Annex A controls; the Statement of Applicability makes an unmonitored guard an audit finding.
- **EU AI Act** - Article 50 transparency **2 Aug 2026**, Annex III high-risk **2 Dec 2027**.

IF YOU REMEMBER ONE THING
Guardrails do not make a model safe. They make a **system accountable**. Put the guard where the damage happens - for agents that is the tool call, which is exactly where the managed guardrail is not looking.